吉林農業大學

本科生毕业论文（设计）

题　　目：基于百度 AI 人脸识别的考勤系统设计与实现

姓　　名：段翔宇　学　　号：2212100323

学　　院：信息技术学院　专　　业：计算机科学与技术

指导教师：徐兴梅　职　　称：教授

2026 年　　4 月　　25 日

基于百度 AI 人脸识别的考勤系统设计与实现

摘　要

随着企事业单位信息化管理程度的不断提高，传统纸质签到、人工核对和事后统计方式已经难以满足考勤管理对实时性、准确性和可追溯性的要求。普通账号密码登录虽然能够完成身份区分，但在代签、补录和地点核验方面仍存在一定漏洞。针对上述问题，本文结合 Spring Boot、Vue、MyBatis-Plus、MySQL 以及百度 AI 人脸识别能力，设计并实现了一套基于百度 AI 人脸识别的考勤系统。系统以员工打卡业务为主线，将人脸比对、考勤设备、部门管理、请假审核、考勤记录、维保记录、公告资讯和交流互动等模块统一组织，形成面向员工、部门主管和管理员的综合考勤管理平台。

本文首先分析了考勤业务的应用背景和国内外研究现状，明确系统在身份核验、数据记录、权限分离和统计管理方面的设计目标；其次对 Spring Boot 后端框架、Vue 前端框架、MyBatis-Plus 持久层、MySQL 数据库、百度 AI 人脸比对接口和高德定位能力进行了说明；随后从功能需求、非功能需求、数据需求和角色权限四个角度展开需求分析，并在总体设计中给出系统架构、功能结构、用例模型、包图、E-R 图、数据库逻辑关系图和核心业务流程。实现部分重点阐述登录鉴权、人脸比对、定位打卡、请假审核、考勤统计和后台管理等关键模块，并结合项目运行页面截图说明系统实现效果。

测试结果表明，系统能够完成用户登录、员工信息管理、考勤设备维护、人脸识别打卡、地理位置采集、请假申请审核、考勤记录统计和公告资讯管理等功能。系统前后端分离结构清晰，接口返回格式统一，数据库表设计能够覆盖主要考勤业务数据，百度 AI 接口的引入提高了考勤身份核验的可靠性。本文的研究与实现对中小型单位考勤管理信息化具有一定参考价值，也为后续加入活体检测、移动端推送和异常考勤预警等功能提供了基础。

关键词：人脸识别；考勤系统；Spring Boot；Vue；百度 AI

Design and Implementation of Attendance System Based on Baidu AI Face Recognition

ABSTRACT

With the continuous improvement of information management in enterprises and institutions, traditional paper-based attendance, manual checking and delayed statistics can no longer meet the requirements of real-time, accurate and traceable attendance management. To solve problems such as proxy check-in, incomplete identity verification and inefficient attendance statistics, this thesis designs and implements an attendance system based on Baidu AI face recognition. The system is developed with Spring Boot, Vue, MyBatis-Plus and MySQL, and integrates Baidu AI face matching and map-based location acquisition. It provides attendance check-in, employee management, department management, leave approval, attendance records, device maintenance, announcements and communication functions for employees, department supervisors and administrators.

The thesis first analyzes the application background and related research of attendance systems, and then introduces the key technologies used in the project. Based on the actual source code and database script, the functional requirements, non-functional requirements, role permissions and data requirements are analyzed. The overall design presents the system architecture, functional structure, use case model, package diagram, E-R model, database logical relationship and core business processes. The detailed implementation focuses on login authentication, face comparison, location-based attendance, leave approval, attendance statistics and management modules, and explains the implementation effect with real running screenshots from the project.

The test results show that the system can complete login authentication, employee information management, attendance device maintenance, face recognition attendance, geographic location collection, leave review, attendance record statistics and announcement management. The front-end and back-end separation structure is clear, the interface response format is unified, and the database tables cover the main business data of attendance management. The integration of Baidu AI improves the reliability of identity verification and provides a practical reference for the informatization of attendance management in small and medium-sized organizations.

KEY WORDS: face recognition; attendance system; Spring Boot; Vue; Baidu AI

目　录

第 1 章 前言.....1
1.1 研究背景.....1
1.2 研究目的及意义.....1
1.3 国内外研究现状.....2
1.4 主要研究内容.....2
1.5 论文组织结构.....3
第 2 章 相关技术介绍.....5
2.1 Spring Boot 框架.....5
2.2 Vue 前端框架.....5
2.3 MyBatis-Plus 与 MySQL.....5
2.4 百度 AI 人脸识别技术.....5
2.5 高德定位与前后端分离.....6
2.6 本章小结.....7
第 3 章 系统需求分析.....8
3.1 业务需求分析.....8
3.2 功能需求分析.....8
3.3 用例分析.....10
3.4 非功能需求分析.....10
3.5 数据需求分析.....11
3.6 本章小结.....11
第 4 章 系统总体设计.....12
4.1 系统总体架构设计.....12
4.2 系统部署结构设计.....13
4.3 系统包结构设计.....13
4.4 人脸识别打卡流程设计.....14
4.5 数据库概念结构设计.....15
4.6 数据库表结构设计.....16
4.7 接口设计.....20
4.8 本章小结.....21
第 5 章 系统详细设计与实现.....22
5.1 登录与权限模块实现.....22
5.2 人脸识别模块实现.....22
5.3 考勤打卡模块实现.....24
5.4 定位采集模块实现.....25

5.5 请假审核模块实现.....25
5.6 考勤设备与维保模块实现.....26
5.7 考勤记录统计模块实现.....27
5.8 系统首页与公共信息模块实现.....27
5.9 后台管理模块实现.....28
5.10 本章小结.....30
第 6 章 系统测试.....31
6.1 测试目的与测试环境.....31
6.2 功能测试.....31
6.3 人脸识别与定位测试.....32
6.4 接口测试.....32
6.5 权限与异常测试.....33
6.6 测试结果分析.....33
6.7 本章小结.....33
第 7 章 总结与展望.....35
7.1 工作总结.....35
7.2 不足之处.....35
7.3 展望.....35
参考文献.....37
附 录.....38
致 谢.....39

第 1 章 前言

1.1 研究背景

考勤管理是企事业单位日常管理中最基础但也最容易产生争议的环节之一。传统考勤方式主要依赖纸质签到、打卡机或人工登记，管理人员需要在月末集中汇总迟到、早退、请假和出勤情况。这种方式虽然实现简单，但对人员身份的确认能力较弱，统计周期长，补录和修改过程缺乏完整痕迹，容易造成数据不一致。随着单位组织规模扩大和工作地点多样化，考勤数据已经不再只是简单的到岗记录，而是与绩效评价、部门协同、设备管理和人员安全等业务密切关联[1-3]。

近年来，人脸识别技术在门禁、支付、校园管理、安防巡检等场景中得到广泛应用。与密码、卡片和指纹相比，人脸具有非接触、采集便捷、用户接受度高等特点，适合与线上考勤系统结合。百度 AI 开放平台提供的人脸比对能力可以将两张人脸图片进行相似度计算，开发者无需自行训练识别模型即可完成身份核验。本文所研究的系统正是基于这一思路，将员工基础头像与打卡时上传的人脸图片进行比对，并结合定位信息和考勤设备信息保存打卡记录[9]。

从实际项目材料看，系统采用 Spring Boot 作为后端主框架，前台和后台页面由 Vue 构建，数据库使用 MySQL 存储业务数据。项目已经包含员工、部门主管、部门、考勤设备、考勤打卡、考勤记录、员工请假、维保记录、公告资讯、交流互动和在线客服等模块，具备较完整的考勤管理业务基础。论文的写作需要围绕该项目实际内容展开，而不是抽象描述一个不存在的系统，因此本文在章节安排、图表设计和实现说明中均以项目源码、数据库脚本和运行页面为依据。

1.2 研究目的及意义

本文的研究目的在于设计并实现一套面向中小型单位的智能考勤管理系统，使考勤过程从单纯记录转变为身份核验、位置采集、数据归档和统计管理相结合的完整业务流程。系统通过员工端完成打卡和请假申请，通过部门主管端完成部门人员相关审核，通过管理员端完成基础数据、设备、资讯和系统日志管理。该设计能够减少人工统计工作量，降低代签和信息误填风险，提高考勤数据的完整性。

从技术意义上看，系统将 Spring Boot 后端、Vue 前端、MyBatis-Plus 数据访问、百度 AI 人脸识别和高德定位能力结合在一起，体现了当前 Web 应用常见的前后端分离开发模式。人脸识别接口并非简单作为页面装饰，而是被嵌入考勤提交前的业务校验流程中：前端先上传或拍摄人脸图片，再调用后端统一接口进行比对，只有匹配成功后才允许继续获取位置并提交打卡数据。该流程能够展示人工智能服务在传统信息管理系统中的具体接入方式。

从应用意义上看，系统能够为单位提供更直观的考勤管理手段。管理员可以维护部门、员工和设备信息，部门主管可以查看本部门人员打卡和请假情况，员工可以在前台完成日常打卡和请假申请。所有数据最终进入数据库，便于后续按照月份、部门、员工和打卡类型进行查询统计。相比单独使用第三方考勤工具，自主开发系统具有字

段可控、业务可扩展和部署方式灵活等优点。

1.3 国内外研究现状

国外较早开展了自动考勤和身份识别系统研究，常见方案包括刷卡考勤、指纹识别考勤、射频识别考勤和基于移动端定位的考勤。随着深度学习的发展，基于卷积神经网络的人脸检测、人脸特征提取和人脸比对技术逐渐成熟，相关算法在公开数据集和实际场景中均表现出较好的识别能力。部分企业级系统还将活体检测、门禁设备和云端人事系统结合起来，实现从人员进入到考勤统计的一体化管理[1-3]。

国内在校园、医院、企业和政务服务等场景中也开展了大量考勤系统研究。相关文献中常见的技术组合包括 Spring Boot 与 Vue 前后端分离、Android 或小程序移动端、MySQL 或 SQL Server 数据库，以及百度、腾讯、阿里等平台提供的人脸识别接口。相比早期以单机 C/S 结构为主的系统，当前研究更加关注 Web 化、移动化、权限管理和数据统计能力。人脸识别的引入则使考勤系统不再完全依赖账号密码，而是增加了生物特征核验环节[4,9-10]。

现有研究仍存在一些不足：一是部分论文只强调识别算法，而对考勤系统的业务数据结构和权限设计描述不足；二是部分系统只展示登录和打卡页面，缺少请假、记录统计和设备维护等完整管理流程；三是人脸识别接口调用与考勤数据保存之间的关系不够清晰，无法体现识别失败时的业务处理。本文在已有项目基础上补充系统结构、数据库表、接口设计和测试过程说明，使论文内容能够更贴近真实软件工程项目。

1.4 主要研究内容

本文围绕基于百度 AI 人脸识别的考勤系统展开，主要研究内容包括以下几个方面。第一，对系统业务需求进行分析，明确管理员、员工和部门主管三类角色的使用边界，并梳理考勤设备、考勤打卡、考勤记录、员工请假、维保记录和公告资讯等功能模块。第二，对项目技术栈进行整理，说明 Spring Boot、Vue、MyBatis-Plus、MySQL、百度 AI 和高德定位在系统中的具体作用。

第三，结合项目数据库脚本设计系统数据模型，重点描述员工表、部门主管表、部门表、考勤设备表、考勤打卡表、考勤记录表、员工请假表和维保记录表之间的联系。第四，对人脸识别打卡流程进行详细说明，解释员工头像与打卡图片如何经后端接口转为 Base64 并调用百度 AI 进行比对，以及前端如何根据相似度阈值决定是否进入定位和提交环节。第五，通过功能测试和运行截图验证系统主要业务流程的可用性。

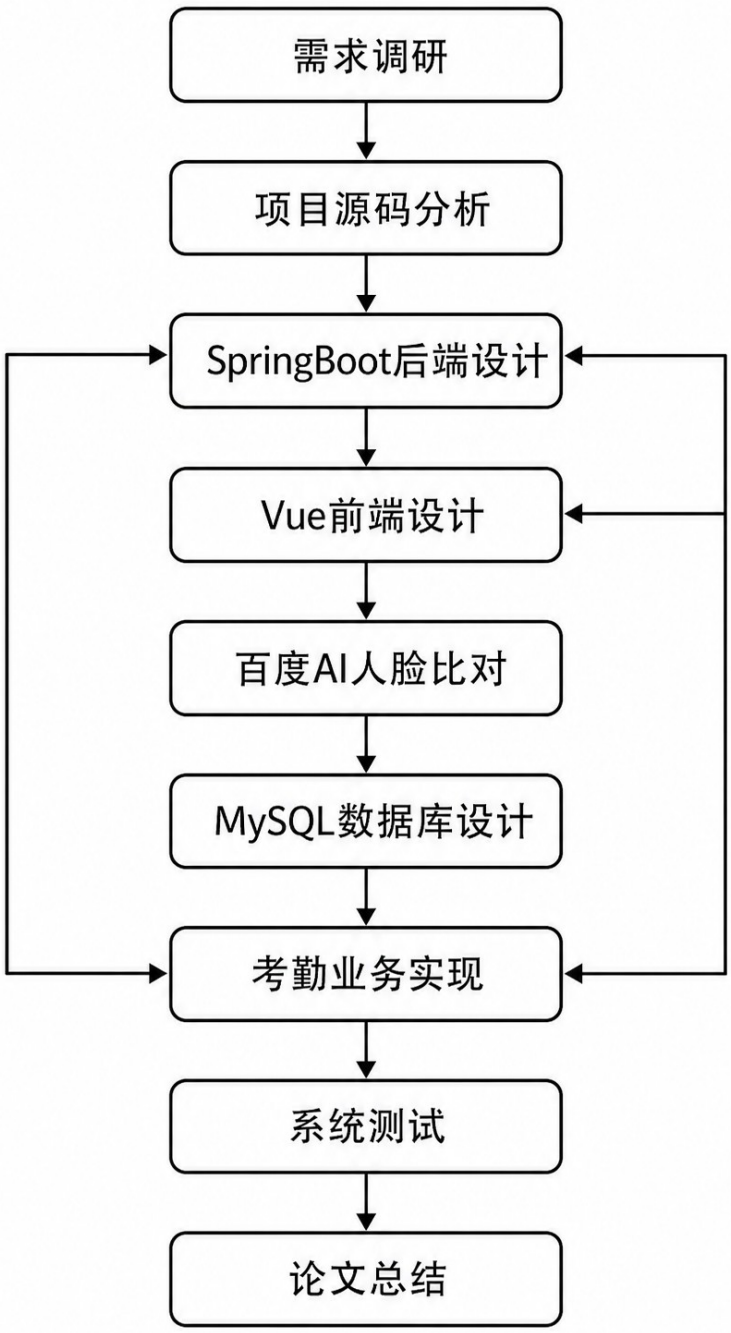

图 1-1 技术路线图

1.5 论文组织结构

本文共分为七章。第 1 章为前言，主要说明研究背景、研究意义、国内外研究现状研究内容和技术路线。第 2 章介绍系统开发所涉及的相关技术，包括 Spring Boot、Vue、MyBatis-Plus、MySQL、百度 AI 人脸识别、高德定位和前后端分离架构。第 3 章进行系统需求分析，从用户角色、功能需求、非功能需求和数据需求角度说明系统应具备的能力。

第 4 章为系统总体设计，给出系统总体架构、部署结构、包图、E-R 图、数据库逻辑关系和主要数据表设计。第 5 章为系统详细设计与实现，重点阐述登录鉴权、人脸

识别打卡、定位采集、请假审核、考勤记录统计和后台管理的实现过程，并给出运行页面截图。第 6 章为系统测试，设计功能测试、接口测试、权限测试和异常测试用例，验证系统的可用性和稳定性。第 7 章为总结与展望，对本文工作进行总结并提出后续改进方向。

第 2 章 相关技术介绍

2.1 Spring Boot 框架

Spring Boot 是当前 Java Web 项目中常用的后端开发框架，其核心特点是自动配置、约定优于配置和内置 Web 容器。本文项目使用 Spring Boot 2.2.2.RELEASE 作为后端基础框架，工程入口、控制器、服务层、实体类、数据访问层和配置文件均遵循典型的 Spring Boot 项目结构。对于考勤系统而言，Spring Boot 主要承担接口暴露、请求参数接收、业务服务调用、数据返回封装和异常处理等任务[4-8]。

项目中的 `controller` 包用于声明前后端交互接口，例如 `KaoqindakaController` 负责考勤打卡记录的分页、详情、新增、修改和删除，`CommonController` 提供文件上传、下载、人脸比对等公共能力。服务层通过 `Service` 和 `ServiceImpl` 组织业务逻辑，数据访问层通过 Mapper 与 MyBatis-Plus 配合完成数据库操作。统一的分层结构使每个模块职责清楚，便于论文从工程角度说明系统实现。

2.2 Vue 前端框架

Vue 是一种渐进式 JavaScript 前端框架，适合构建页面交互频繁的管理系统。本项目的前台和后台页面均基于 Vue 开发，前台用于员工登录、查看公告、查看考勤设备、提交考勤打卡和请假申请，后台用于管理员进行人员、部门、设备、记录和系统配置管理。页面通过路由组织不同功能模块，并通过 HTTP 请求与后端接口交互[10-12,18-19]。

在人脸识别打卡页面中，Vue 组件维护 `ruleForm` 表单数据和 `faceMatchFlag` 校验状态。用户上传或拍摄人脸图片后，页面先调用后端 `matchFace` 接口进行人脸比对，若返回相似度大于阈值，则允许继续获取当前位置并提交考勤数据。该过程体现了前端状态控制与后端业务校验的协同关系，也是系统区别于普通考勤表单的关键点。

2.3 MyBatis-Plus 与 MySQL

MyBatis-Plus 是在 MyBatis 基础上封装的增强工具，能够减少常见 CRUD 代码量。项目中的实体类通过 `@TableName` 与数据库表绑定，Mapper 继承 MyBatis-Plus 基础接口，服务层再调用分页查询、条件查询、插入、更新和批量删除等方法。对于考勤系统这种表结构较多、管理页面较多的项目而言，MyBatis-Plus 可以明显降低重复代码数量[14-17]。

数据库采用 MySQL 5.7，字符集使用 utf8mb4，能够满足中文字段、图片路径、日志内容和长文本公告的存储需求。系统数据库名称为 `springboot5s7063d3`，主要业务表包括部门、部门主管、员工、考勤设备、考勤打卡、考勤记录、员工请假、维保记录、系统用户、令牌、公告资讯和系统日志等。数据库表字段多采用中文拼音命名，与项目实体类字段保持一致，便于从源码追踪到数据库。

2.4 百度 AI 人脸识别技术

百度 AI 人脸识别开放平台提供人脸检测、人脸比对、人脸搜索等能力。本文项

目使用的是人脸比对能力，即将两张人脸图片提交到接口，由平台返回相似度分数。系统并不在本地训练模型，而是通过后端 SDK 调用云端能力。这样既降低了算法开发门槛，也减少了服务器端模型部署和维护成本，适合本科毕业设计场景下的工程实现[9]。

项目在 `pom.xml` 中引入 `com.baidu.aip:java-sdk:4.4.1` 依赖，在 `CommonUtil.matchFace` 方法中读取系统配置表中的 APIKey 和 SecretKey，初始化 `AipFace` 客户端后，将员工头像和打卡人脸图片从上传目录读取并转为 Base64，构造 `MatchRequest` 后调用 `client.match`。接口返回结果中包含 `score` 字段，前端以 60 分作为是否匹配成功的判断依据。论文在实现部分会对这一流程作进一步说明。

2.5 高德定位与前后端分离

考勤业务除了确认“谁在打卡”，还需要尽量记录“在哪里打卡”。项目在前台打卡页面中使用浏览器 Geolocation API 获取经纬度，并调用高德地图反向地理编码将经纬度转换为地址文本。当浏览器定位失败时，页面会尝试使用 IP 定位获取城市级位置。虽然该方式不能完全替代硬件门禁定位，但能够为考勤记录提供更丰富的环境信息。

系统采用前后端分离结构，前端页面负责展示、表单校验、图片上传、位置获取和用户提示，后端接口负责身份校验、业务数据处理、数据库操作和人脸比对接口调用。前后端之间主要通过 JSON 数据进行交互，接口返回统一使用 `R.ok()` 或 `R.error()` 格式。这种结构降低了页面和业务逻辑之间的耦合度，也便于后续将员工端迁移到移动端或小程序端。

表 2-1 对本文项目所使用的主要技术进行汇总。该表不是简单罗列软件名称，而是结合项目源码说明各项技术在系统中的实际承担职责。

表 2-1 系统主要技术栈

技术名称	版本或组件	在系统中的作用
Spring Boot	2.2.2.RELEASE	构建后端接口、组织控制器和业务服务
Vue	前台/后台页面	实现员工端与管理端页面交互
MyBatis-Plus	2.3	封装数据库增删改查和分页查询
MySQL	5.7	存储员工、部门、考勤、请假和日志数据
Baidu AI SDK	java-sdk 4.4.1	完成人脸图片相似度比对
高德地图	Geolocation/ Geocoder	获取打卡经纬度与地址信息
Shiro/Token	项目权限组件	实现登录鉴权和访问控制

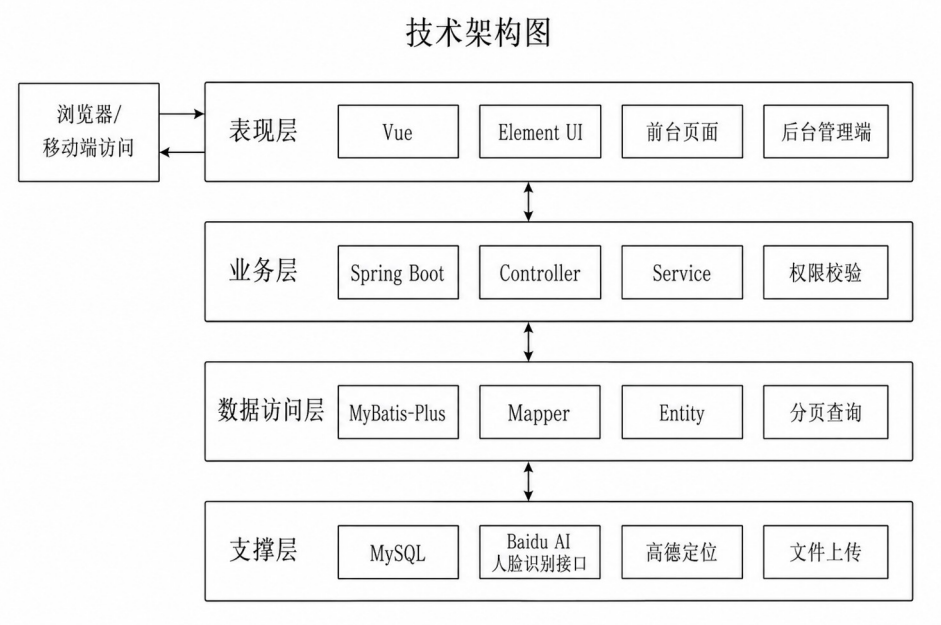

图 2-1 技术架构图

2.6 本章小结

本章围绕系统开发所涉及的关键技术进行了说明。Spring Boot 和 Vue 构成系统的前后端基础，MyBatis-Plus 与 MySQL 支撑数据持久化，百度 AI 人脸识别接口提供身份核验能力，高德定位补充打卡地点信息。上述技术并不是独立堆叠，而是在考勤打卡这一核心业务流程中共同发挥作用，为后续需求分析和系统设计奠定了技术基础。

第 3 章 系统需求分析

3.1 业务需求分析

基于百度 AI 人脸识别的考勤系统面向单位内部考勤管理场景，系统需要解决的主要问题包括员工身份核验、考勤设备维护、部门人员管理、打卡记录保存、请假流程审批和考勤结果统计。与普通信息管理系统相比，本系统的核心业务并不是单纯录入表单，而是通过人脸识别和定位采集增强打卡记录的可信度。因此需求分析必须从实际业务过程出发，明确每类用户在不同环节中的职责[1-3]。

员工是考勤数据的主要产生者，需要能够登录系统、查看公告、查看考勤设备、提交上班或下班打卡、上传人脸图片、查看自己的考勤记录并提交请假申请。部门主管需要关注本部门员工的请假和考勤情况，能够查看部门相关数据并进行必要审核。管理员则负责系统基础数据维护，包括员工、部门主管、部门、考勤设备、考勤记录、维保记录、公告资讯、交流互动和系统日志等模块。

表 3-1 给出了系统主要用户角色及其业务职责。该表用于说明权限划分的依据，后续功能结构图和用例图均围绕这些角色展开。

表 3-1 系统用户角色分析

角色	主要职责	典型操作
管理员	负责系统基础数据、权限和公共信息管理	管理员工、部门、设备、公告、日志和考勤记录
员工	完成个人考勤、请假和信息查询	登录、打卡、人脸校验、请假、查看记录
部门主管	负责部门员工考勤和请假相关管理	查看部门数据、审核请假、核对考勤情况
游客/未登录用户	只能访问公开页面或进入登录入口	浏览首页、进入登录或注册页面

3.2 功能需求分析

功能需求的设计不能只停留在页面增删改查层面，核心在于把考勤业务中的人员身份、部门归属、打卡时间、打卡地点和审核责任组织成可追踪的数据链。考勤打卡模块在系统中承担了承上启下的作用，既要服务员工端的日常操作，也要支撑部门主管和管理员进行后续核验。论文在描述该部分时以真实项目结构为基础，将页面、接口、实体类和数据库表之间的对应关系一并说明，避免只讲功能名称而缺少实现依据。

从项目代码来看，考勤打卡模块并不是孤立模块，而是与登录会话、权限控制、图片上传、分页查询和系统日志共同工作。人脸识别、定位和表单提交被拆分为连续但相互约束的步骤。因此在实现时需要关注两个方面：一是业务字段要能够完整记录考勤发生的上下文，二是接口返回要保持统一格式，便于前端页面在校验、提示和跳转

时进行稳定处理。这样的处理方式使系统具有较好的可维护性，也便于后期增加统计报表、异常提醒等扩展功能。

根据项目源码和数据库脚本，系统功能可以分为基础管理、考勤业务、请假业务、信息发布和系统支撑五类。基础管理包括员工管理、部门主管管理、部门管理和用户管理，是系统其他业务开展的前提。考勤业务包括考勤设备维护、考勤打卡和考勤记录统计，是本系统的核心。请假业务用于处理员工因事、因病等情况产生的非正常出勤记录。信息发布与交流互动模块用于提高系统的通知和沟通能力。

图 3-1 展示了系统功能结构。图中将管理员、员工和部门主管涉及的功能统一放在同一结构中，便于观察模块之间的归属关系。

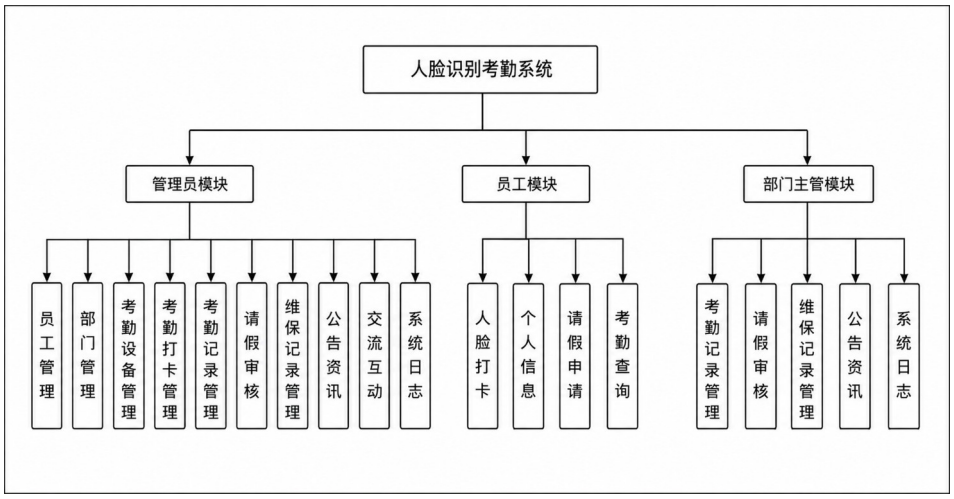

图 3-1 系统功能结构图

表 3-2 进一步列出各功能模块的具体需求。每个模块均对应项目中的控制器、实体类和数据库表，因此该表也为后续详细设计提供了索引。

表 3-2 系统功能需求表

模块	功能需求	对应数据或接口
登录与权限	支持管理员、员工和部门主管登录，依据角色加载菜单和数据范围	users、token、session
员工管理	维护员工工号、姓名、部门、头像、联系方式等信息	yuangong
部门管理	维护部门名称和主管工号，保证部门信息唯一	bumen
考勤设备	维护设备名称、图片、位置和所属部门	kaoqinshebei
考勤打卡	上传人脸图片，比对成功后记录打卡类型、时间和地址	kaoqindaka、matchFace
考勤记录	按月份汇总迟到、早退、旷工、请假和实际出勤数据	kaoqinjilu
员工请假	提交请假申请并由主管或管理员审核	yuangongqingjia
维保记录	记录考勤设备维护时间、内容和处理情况	weibaojilu

公告资讯	发布系统公告和考勤管理通知	news

3.3 用例分析

用例分析用于描述用户与系统之间的交互关系。本系统中管理员的用例范围最广，负责各类基础数据维护和系统管理；员工的用例集中在个人业务操作；部门主管的用例则介于二者之间，主要围绕部门员工考勤和请假审核展开。用例分析不仅说明“用户能做什么”，也能够为菜单权限、接口权限和数据过滤提供依据。

图 3-2 为系统用例图。图中将三类角色与其常用功能连接起来，展示了系统的主要业务边界。

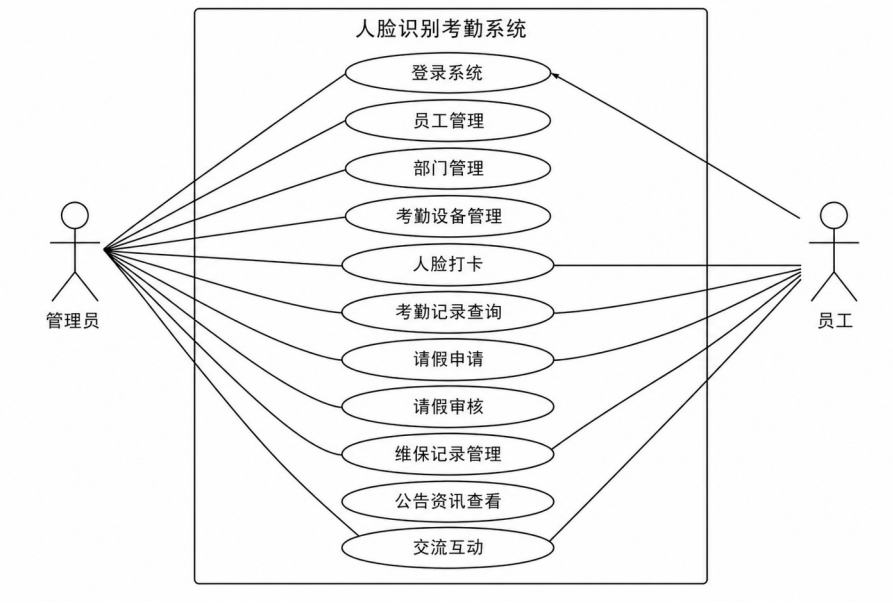

图 3-2 系统用例图

3.4 非功能需求分析

除功能需求外，系统还需要满足一定的非功能需求。首先是易用性需求，考勤打卡属于高频操作，页面必须尽量减少员工填写字段，能够自动带出员工工号、姓名、部门和主管工号等信息。其次是可靠性需求，系统需要在上传图片、人脸比对、定位获取和数据提交之间保持清晰的先后顺序，避免识别失败仍然提交记录。再次是可维护性需求，后台各模块应保持一致的接口风格和代码结构。

安全性也是本系统的重要需求。系统需要通过登录态区分不同用户，避免未授权用户访问后台接口。人脸识别所需的 APIKey 和 SecretKey 应保存在配置表中，由系统在服务端读取，不能在前端页面中直接暴露。打卡图片、头像和设备图片以文件路径形式保存到数据库，实际文件存放于服务器上传目录，便于统一管理。

表 3-3 从性能、安全、可维护和兼容等角度归纳系统非功能需求。

表 3-3 系统非功能需求表

需求类别	具体要求	说明
性能	常规列表分页查询响应及时，人	项目使用分页工具减少一次性加载数据

	脸比对接口在可接受时间内返回	量
安全	接口应结合登录态和角色权限控制访问范围	配置密钥只在服务端读取，避免前端暴露
可靠性	人脸校验失败时不允许直接提交打卡数据	前端使用 faceMatchFlag 控制提交状态
易用性	员工基础信息自动带出，减少重复填写	通过 session 接口获取当前用户信息
可维护性	控制器、服务层、实体类和 Mapper 结构保持一致	便于后续扩展新业务模块
兼容性	支持主流浏览器访问前台和后台页面	前端基于 Vue 构建，接口采用 JSON 交互

3.5 数据需求分析

考勤系统的数据需求具有明显的业务关联性。员工表记录人员基础信息，部门表记录组织结构，部门主管表记录管理者信息，考勤设备表记录设备位置和归属部门，考勤打卡表记录每一次打卡行为，考勤记录表对月度出勤进行汇总，员工请假表记录请假申请和审核状态，维保记录表记录设备维护情况。上述表共同支撑系统完整业务流程。

在数据设计上，系统需要尽量保存业务发生时的关键上下文。例如考勤打卡表除了保存员工工号，还保存员工姓名、部门名称、主管工号、打卡类别、打卡时间、人脸识别图片、经纬度和地址。这样即使后续人员基础信息发生变化，历史打卡记录仍然保留当时的业务信息。请假表和考勤记录表也采用类似思路，将统计结果和审核信息直接记录，便于查询和导出。

3.6 本章小结

本章从业务、角色、功能、非功能和数据五个方面对系统需求进行了分析。需求分析表明，本系统不仅需要完成常规管理系统的增删改查，还需要围绕人脸识别打卡形成一条完整的业务链路。下一章将在需求分析基础上进行总体设计，说明系统架构、数据库结构、包结构和核心流程。

第 4 章 系统总体设计

4.1 系统总体架构设计

系统总体架构采用典型的前后端分离模式。前端由员工端和后台管理端组成，负责页面展示、表单校验、图片上传、定位采集和用户交互；后端由 Spring Boot 提供 REST 风格接口，负责业务处理、权限校验、数据持久化和第三方接口调用；数据库采用 MySQL 保存业务数据；外部服务主要包括百度 AI 人脸识别接口和高德地图定位服务[10,18-19]。

总体架构的设计不能只停留在页面增删改查层面，核心在于把考勤业务中的人员身份、部门归属、打卡时间、打卡地点和审核责任组织成可追踪的数据链。前后端分离架构在系统中承担了承上启下的作用，既要服务员工端的日常操作，也要支撑部门主管和管理员进行后续核验。论文在描述该部分时以真实项目结构为基础，将页面、接口、实体类和数据库表之间的对应关系一并说明，避免只讲功能名称而缺少实现依据。

从项目代码来看，前后端分离架构并不是孤立模块，而是与登录会话、权限控制、图片上传、分页查询和系统日志共同工作。前端主要负责交互与状态，后端主要负责接口、权限、第三方服务调用和数据库操作。因此在实现时需要关注两个方面：一是业务字段要能够完整记录考勤发生的上下文，二是接口返回要保持统一格式，便于前端页面在校验、提示和跳转时进行稳定处理。这样的处理方式使系统具有较好的可维护性，也便于后期增加统计报表、异常提醒等扩展功能。

图 4-1 展示了系统总体架构。该图将浏览器端、业务服务端、数据库和外部服务放在同一层次中描述，能够清楚体现系统运行时的数据流向。

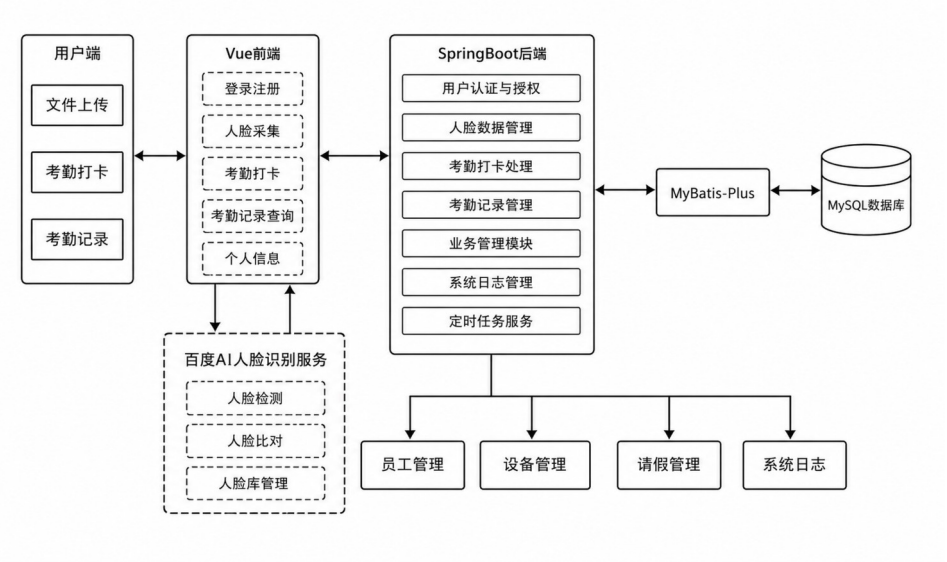

图 4-1 系统总体架构图

系统运行时，用户首先通过前端页面发起登录、查询或提交请求。普通业务请求由后端控制器转交服务层处理，服务层通过 Mapper 访问 MySQL 数据库。涉及人脸识别的请求需要额外读取上传目录中的两张图片，并调用百度 AI SDK 获取相似度结果。涉及定位的请求主要在前端完成经纬度和地址采集，最终与打卡记录一起提交到后端

保存。

4.2 系统部署结构设计

项目部署结构可分为浏览器、Web 服务、数据库服务和第三方接口四部分。浏览器端访问前台或后台静态资源，Web 服务端运行 Spring Boot 应用并提供接口，数据库服务保存业务数据，第三方接口提供人脸比对和地址解析能力。由于项目采用 Maven 构建，后端可以打包为可运行的 Spring Boot 应用，也可以根据部署环境发布到 Web 容器中。

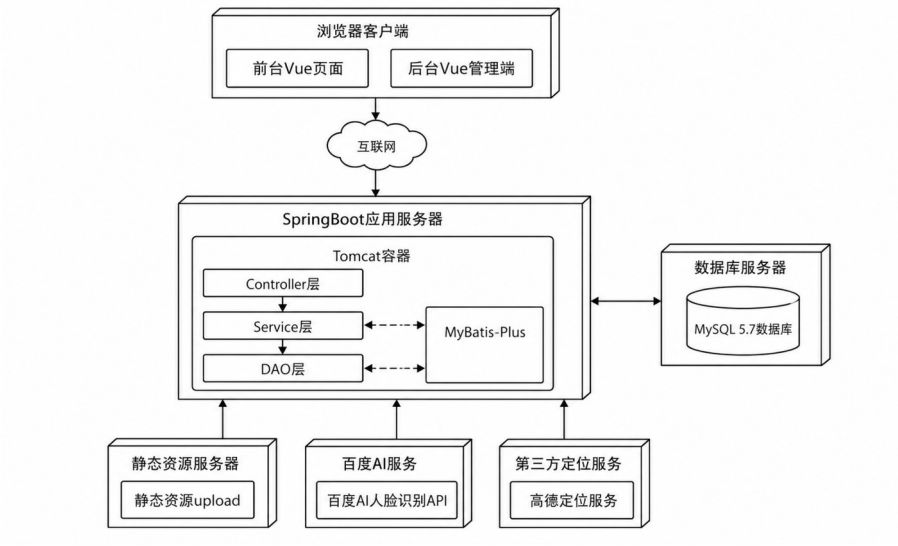

图 4-2 系统部署结构图

在实际部署中，前端静态资源可以与后端服务同域部署，也可以由独立 Web 服务器托管。数据库连接信息写入 `application.yml` 配置文件，上传文件目录位于后端静态资源目录下。人脸识别接口的密钥信息通过系统配置表维护，便于在不修改代码的情况下进行调整。

4.3 系统包结构设计

包结构设计用于说明项目代码的组织方式。项目后端主要包含 controller、service、dao、entity、utils、config、annotation 和 interceptor 等包；前端页面按照业务模块划分目录，例如员工、部门、考勤设备、考勤打卡、员工请假、考勤记录等。清晰的包结构能够降低模块之间的耦合，使开发者在定位问题时能够快速找到相关代码。

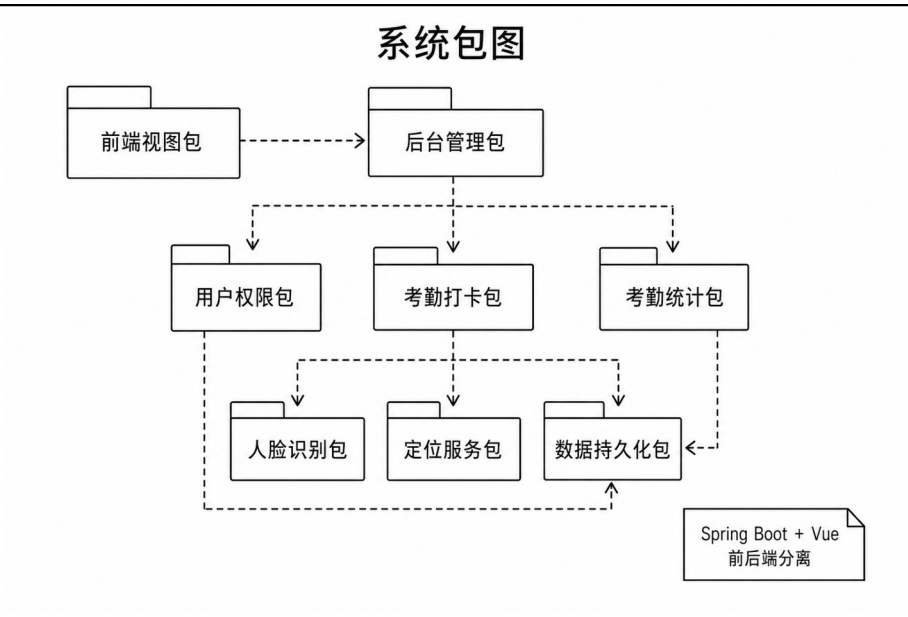

图 4-3 系统包图

从包图可以看出，前端视图包通过 HTTP 请求访问后台管理包和业务接口包，业务接口再调用用户权限包、考勤打卡包、考勤统计包等模块。考勤打卡包同时依赖人脸识别包、定位服务和数据持久化包，是系统中业务关联最多的模块。这样的结构与项目源码基本一致，也有助于将论文中的设计说明与真实工程文件对应起来。

4.4 人脸识别打卡流程设计

人脸识别打卡是系统最核心的业务流程。员工进入打卡页面后，系统自动填充当前时间和员工基础信息，员工选择打卡类别并上传人脸图片。前端先调用后端 `matchFace` 接口进行人脸比对，后端从上传目录读取员工头像和打卡图片，调用百度 AI 接口获取相似度分数。当前端判断相似度大于阈值后，才将 `faceMatchFlag` 设置为成功状态，并继续执行定位和提交[9]。

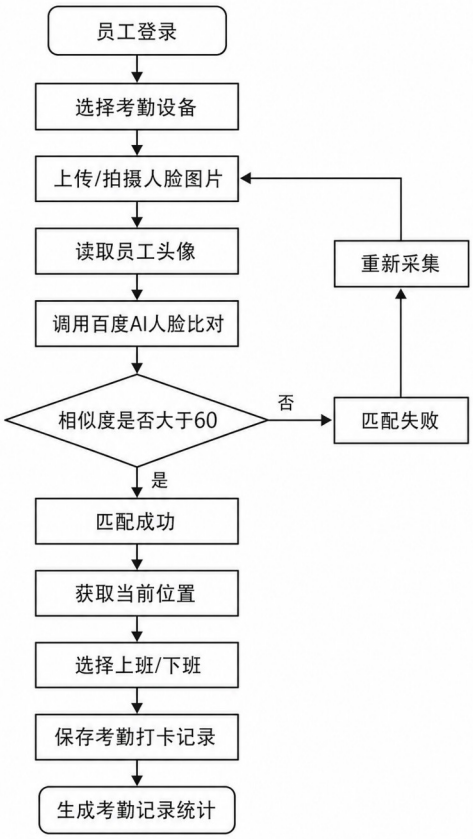

图 4-4 人脸识别打卡流程图

该流程的关键在于把识别、定位和提交三个环节拆开处理。人脸识别失败时，系统只提示失败，不保存打卡数据；识别成功后，系统再尝试获取当前位置，如果高精度定位失败，则回退到 IP 定位。最终保存的打卡记录包含打卡类别、打卡时间、员工信息、人脸图片、经纬度和地址，为后续考勤核查提供依据。

4.5 数据库概念结构设计

数据库概念结构设计用于从业务实体角度描述系统数据关系。系统中的核心实体包括员工、部门主管、部门、考勤设备、考勤打卡、考勤记录、员工请假和维保记录。其中员工与考勤打卡、考勤记录、员工请假之间存在一对多关系；部门与员工、部门主管、设备之间存在关联；考勤设备与打卡记录和维保记录之间存在关联。

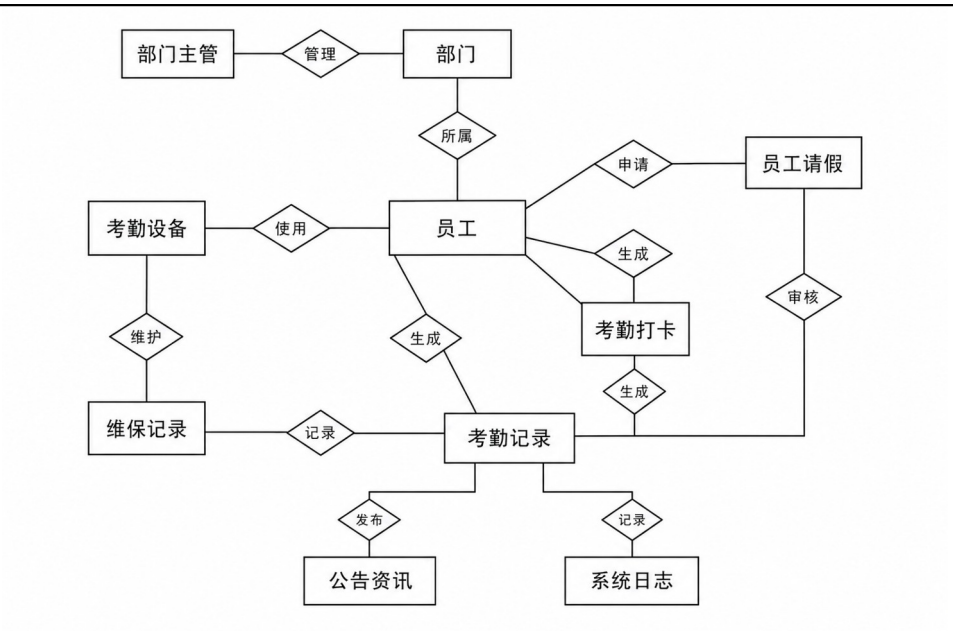

图 4-5 系统 E-R 图

E-R 图体现了系统业务数据的基本关系。由于项目数据库中部分表未强制声明外键，论文中的关系主要依据业务字段建立，例如员工工号、部门名称、主管工号和设备名称。这样既符合项目实际，也能说明系统在业务层面如何通过字段进行关联查询和数据约束。

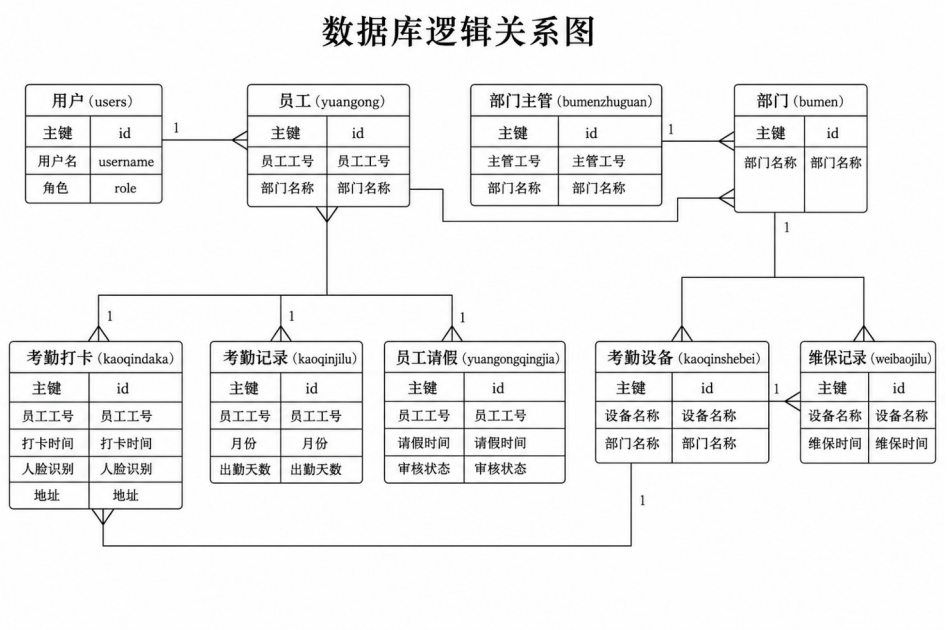

图 4-6 数据库逻辑关系图

4.6 数据库表结构设计

数据库表结构是系统实现的基础。由于本系统模块较多，如果只给出总表清单，读者难以理解每张表的业务含义。因此本节按照核心业务顺序分别说明主要数据表，每个表前均给出用途说明，再列出关键字段。

部门表用于维护单位内部组织结构，部门名称具有唯一性，主管工号字段用于记录该部门对应的管理人员。该表为员工、设备和考勤统计提供组织维度。

表 4-1 部门表（bumen）

字段名	类型	说明
id	bigint	主键，自增
addtime	timestamp	创建时间
bumenmingcheng	varchar(200)	部门名称，唯一
zhuguangonghao	varchar(200)	主管工号

员工表用于保存员工基础信息，是系统中多数业务数据的来源。员工打卡、请假和考勤统计均需要引用员工工号、姓名、头像和部门名称等字段。

表 4-2 员工表（yuangong）

字段名	类型	说明
id	bigint	主键，自增
yuangonggonghao	varchar(200)	员工工号，唯一
mima	varchar(200)	登录密码
yuangongxingming	varchar(200)	员工姓名
touxiang	longtext	员工头像，用于人脸比对基础图
xingbie	varchar(200)	性别
bumenmingcheng	varchar(200)	部门名称
zhuguangonghao	varchar(200)	主管工号
shouji	varchar(200)	手机号码
status	int	账号状态

部门主管表用于保存部门管理者信息。该表与员工表结构相近，但业务角色不同，主要用于部门数据查看、请假审核和本部门考勤管理。

表 4-3 部门主管表（bumenzhuguan）

字段名	类型	说明
id	bigint	主键，自增
zhuguangonghao	varchar(200)	主管工号，唯一
mima	varchar(200)	登录密码
zhuguanxingming	varchar(200)	主管姓名
touxiang	longtext	头像

xingbie	varchar(200)	性别
bumenmingcheng	varchar(200)	部门名称
jinglidianhua	varchar(200)	联系电话
status	int	账号状态

考勤设备表记录设备名称、图片、位置和所属部门。员工在打卡时可选择对应设备，管理员也可通过维保记录维护设备状态。

表 4-4 考勤设备表（kaoqinshebei）

字段名	类型	说明
id	bigint	主键，自增
kaoqinshebei	varchar(200)	考勤设备名称
tupian	longtext	设备图片
shebeiweizhi	varchar(200)	设备位置
bumenmingcheng	varchar(200)	所属部门
addtime	timestamp	创建时间

考勤打卡表是系统的核心业务表，用于保存每次打卡形成的原始记录。表中不仅记录员工和部门信息，还记录人脸识别图片、经纬度和地址，使后续核查具有更完整依据。

表 4-5 考勤打卡表（kaoqindaka）

字段名	类型	说明
id	bigint	主键，自增
kaoqinshebei	varchar(200)	考勤设备
dakaleibie	varchar(200)	打卡类别，上班或下班
dakashijian	datetime	打卡时间
yuangonggonghao	varchar(200)	员工工号
yuangongxingming	varchar(200)	员工姓名
bumenmingcheng	varchar(200)	部门名称
zhuguangonghao	varchar(200)	主管工号
renlianshibie	longtext	打卡人脸图片路径
longitude	float	经度

latitude	float	纬度
fulladdress	varchar(200)	详细地址

考勤记录表用于保存月度统计结果，字段包括迟到、早退、旷工、出差、请假和实际出勤天数等。该表面向管理统计，能够减少每次查询都重新计算原始打卡记录的压力。

表 4-6 考勤记录表（kaoqinjilu）

字段名	类型	说明
id	bigint	主键，自增
yuefen	varchar(200)	统计月份
yuangonggonghao	varchar(200)	员工工号
yuangongxingming	varchar(200)	员工姓名
bumenmingcheng	varchar(200)	部门名称
chidaocishu	int	迟到次数
zaotuicishu	int	早退次数
kuanggongcishu	int	旷工次数
qingjiatianshu	int	请假天数
shijichuqintianshu	int	实际出勤天数
beizhu	longtext	备注

员工请假表用于记录请假申请、请假天数、申请时间和审核状态。该表与考勤记录表共同反映员工月度出勤情况，是考勤统计不可缺少的数据来源。

表 4-7 员工请假表（yuangongqingjia）

字段名	类型	说明
id	bigint	主键，自增
qingjiabianhao	varchar(200)	请假编号
qingjialeixing	varchar(200)	请假类型
qingjiashiyou	longtext	请假事由
qingjiatianshu	int	请假天数
yuangonggonghao	varchar(200)	员工工号
yuangongxingming	varchar(200)	员工姓名

zhuguangonghao	varchar(200)	主管工号
qingjiashijian	datetime	请假时间
sfsh	varchar(200)	是否审核
shhf	longtext	审核回复

维保记录表用于记录考勤设备维护情况。考勤设备属于考勤业务的基础设施，记录维护信息有助于说明设备异常时考勤数据可能受到的影响。

表 4-8 维保记录表（weibaojilu）

字段名	类型	说明
id	bigint	主键，自增
kaoqinshebei	varchar(200)	考勤设备
weibaoshijian	datetime	维保时间
weibaoneirong	longtext	维保内容
weibaoren	varchar(200)	维保人员
beizhu	longtext	备注
addtime	timestamp	创建时间

4.7 接口设计

系统接口采用模块化路径设计，前端页面按照业务模块调用后端控制器。通用接口包括登录、上传、下载和人脸比对，业务接口包括员工、部门、设备、打卡、请假、记录和资讯等模块的分页、详情、新增、修改和删除。统一接口风格使前端页面能够复用请求方法，也便于后期维护。

表 4-9 主要接口设计表

接口路径	请求方式	功能说明
/matchFace	GET	对两张人脸图片进行相似度比对
/kaoqindaka/page	GET	分页查询考勤打卡记录
/kaoqindaka/add	POST	前台新增考勤打卡记录
/kaoqindaka/update	POST	修改考勤打卡记录
/yuangongqingjia/add	POST	员工提交请假申请
/kaoqinjilu/page	GET	分页查询考勤记录
/kaoqinshebei/page	GET	分页查询考勤设备

/file/upload	POST	上传头像、人脸图片或设备图片

4.8 本章小结

本章完成了系统总体设计，分别从架构、部署、包结构、业务流程、数据库概念结构、逻辑关系、表结构和接口设计等方面进行了说明。总体设计表明，系统以考勤打卡为核心，将人脸识别、定位、人员组织和管理统计连接起来，形成了较完整的软件结构。

第 5 章 系统详细设计与实现

5.1 登录与权限模块实现

系统登录模块负责识别当前用户身份，并根据角色加载不同的页面和接口权限。管理员通过后台管理端登录，员工和部门主管可以通过前台或对应入口完成登录。登录成功后，系统保存用户会话信息，前端在需要当前用户资料时调用 session 接口获取员工工号、姓名、部门名称、主管工号和头像等字段。

权限模块的实现重点不在于单个页面的样式，而在于保证不同角色看到的数据范围不同。例如考勤打卡分页接口会根据 session 中的用户身份进行条件设置：员工只能查看自己的打卡记录，部门主管则根据主管工号查看相关部门数据，管理员可以查看全部数据。这种处理方式符合考勤业务中的分级管理要求。

图 5-1 前台登录页面

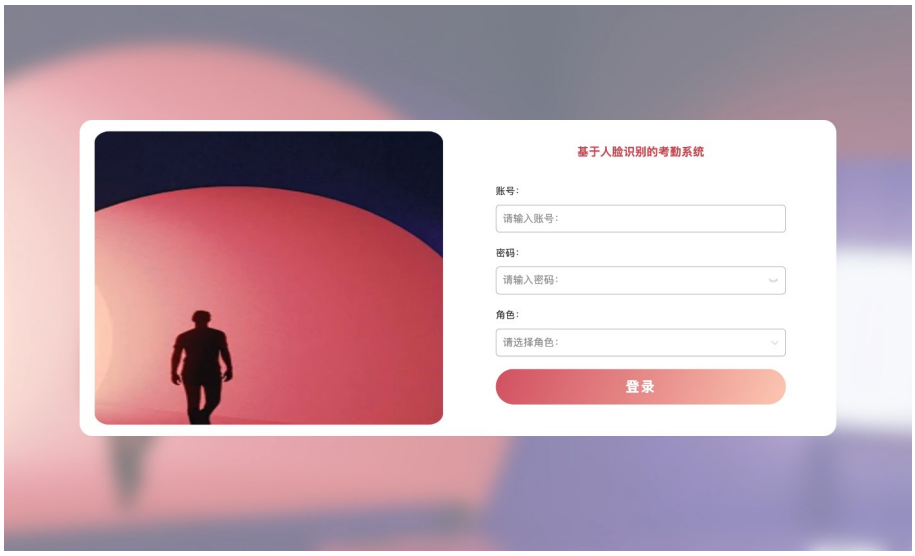

图 5-2 后台登录页面

5.2 人脸识别模块实现

人脸识别模块由前端打卡页面和后端公共接口共同完成。前端页面上传打卡人脸图片后，将当前用户头像和打卡图片路径作为参数传给 `matchFace` 接口。后端接口本

身只负责接收参数并调用工具方法，具体识别逻辑写在 `CommonUtil.matchFace` 中。这样的设计使识别能力成为公共服务，后续如果其他模块也需要人脸比对，可以复用该接口。

后端识别逻辑首先从配置表读取百度 AI 的访问密钥，并初始化 `AipFace` 客户端；随后从项目静态上传目录读取两张图片，将图片转为 Base64；最后构造两个 `MatchRequest` 对象调用百度 AI 的 `match` 方法。若接口返回错误码不为 0，系统返回相似度为 0；若调用成功，则从返回结果中提取 score 字段并返回给前端[9]。

详细实现的设计不能只停留在页面增删改查层面，核心在于把考勤业务中的人员身份、部门归属、打卡时间、打卡地点和审核责任组织成可追踪的数据链。考勤打卡页面在系统中承担了承上启下的作用，既要服务员工端的日常操作，也要支撑部门主管和管理员进行后续核验。论文在描述该部分时以真实项目结构为基础，将页面、接口、实体类和数据库表之间的对应关系一并说明，避免只讲功能名称而缺少实现依据。

从项目代码来看，考勤打卡页面并不是孤立模块，而是与登录会话、权限控制、图片上传、分页查询和系统日志共同工作。页面中的按钮状态、表单字段和接口调用顺序共同决定打卡流程是否可靠。因此在实现时需要关注两个方面：一是业务字段要能够完整记录考勤发生的上下文，二是接口返回要保持统一格式，便于前端页面在校验、提示和跳转时进行稳定处理。这样的处理方式使系统具有较好的可维护性，也便于后期增加统计报表、异常提醒等扩展功能。

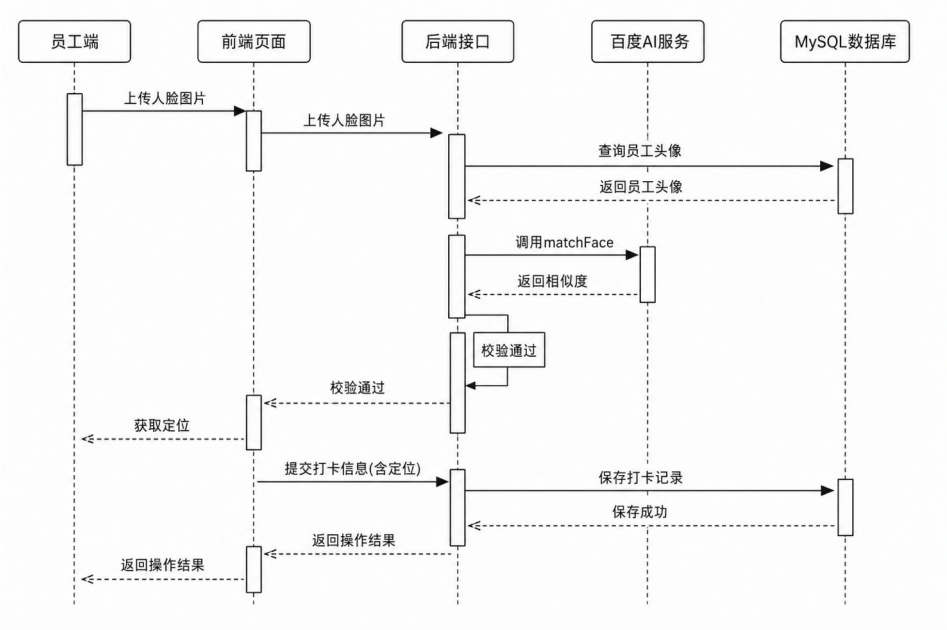

图 5-3 人脸识别打卡时序图

程序 5-1 人脸比对接口核心代码

```
@RequestMapping("/matchFace")
@IgnoreAuth
public R matchFace(String face1, String face2, HttpServletRequest request) {
    return CommonUtil.matchFace(face1, face2);
}
```

前端收到比对结果后，并不会立即提交表单，而是判断返回分数是否大于 60。若分数满足要求，则将 `faceMatchFlag` 设置为 true 并提示匹配成功；若分数不足，则提示匹配失败并保持不可提交状态。该处理能够避免用户在未通过身份核验的情况下直接保存考勤记录。

5.3 考勤打卡模块实现

考勤打卡页面是系统中交互最复杂的页面之一。页面包含考勤设备、打卡类别、打卡时间、员工工号、员工姓名、部门名称、主管工号、人脸识别图片和地址等字段。系统创建页面时会自动设置当前时间，并通过当前用户 session 自动填入员工相关信息，减少用户手工录入。

图 5-4 为前台考勤打卡页面运行效果。可以看到页面中的员工信息、打卡类别、人脸识别和地址信息都围绕一次打卡记录展开，符合数据库中 `kaoqindaka` 表的字段设计。

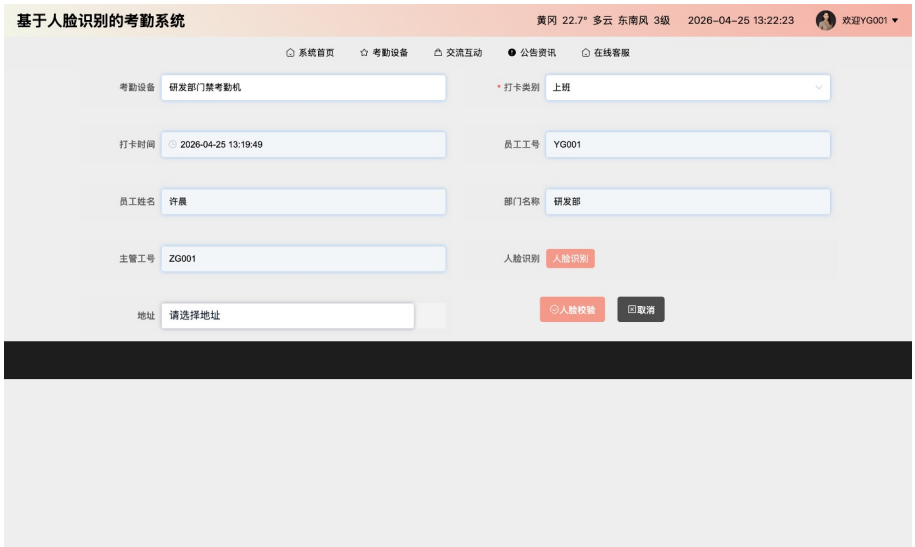

图 5-4 前台考勤打卡页面

打卡提交分为两个阶段。第一阶段是人脸校验，系统校验图片是否上传，调用后端比对接口，并根据相似度决定是否通过。第二阶段是表单提交，只有当 `faceMatchFlag` 为 true 时，系统才执行表单校验并调用 `kaoqindaka/add` 或 `kaoqindaka/update` 接口保存记录。分阶段处理使页面逻辑更清晰，也便于定位异常。

程序 5-2 前端人脸校验与提交控制代码

```
let params = {
    face1: this.userface.replace("upload/", ""),
    face2: this.ruleForm.renlianshibie.replace("upload/", "")
}
const res = await this.$http.get("matchFace", { params: params });
if (res.data && res.data.code == 0 && res.data.score > 60) {
    this.faceMatchFlag = true;
    this.$message({ message: "□□□□", type: "success" });
} else {
    this.faceMatchFlag = false;
    this.$message.error("□□□□");
```

}

5.4 定位采集模块实现

定位采集模块用于补充打卡地点信息。项目中前端页面在通过人脸识别后调用 `forceGetCurrentPosition` 方法，优先使用浏览器 Geolocation API 获取经纬度，再通过高德地图 Geocoder 插件将经纬度转换为格式化地址。若浏览器无法提供精确位置，系统会进入 IP 定位回退流程，至少保存城市级位置。

这种设计体现了一定容错性。考勤场景中，定位能力会受到浏览器权限、网络环境和设备设置影响，如果只依赖一种定位方式，可能导致用户无法完成打卡。通过“高精度定位优先、IP 定位兜底”的方式，系统既尽量获取准确地址，也避免定位失败直接阻断后续业务。

5.5 请假审核模块实现

请假审核模块用于处理员工因事、因病或其他原因产生的非正常出勤。员工进入请假页面后填写请假编号、请假类型、请假事由、请假天数、员工信息、主管工号和请假时间。提交后，请假数据保存到 `yuangongqingjia` 表，等待主管或管理员进行审核。

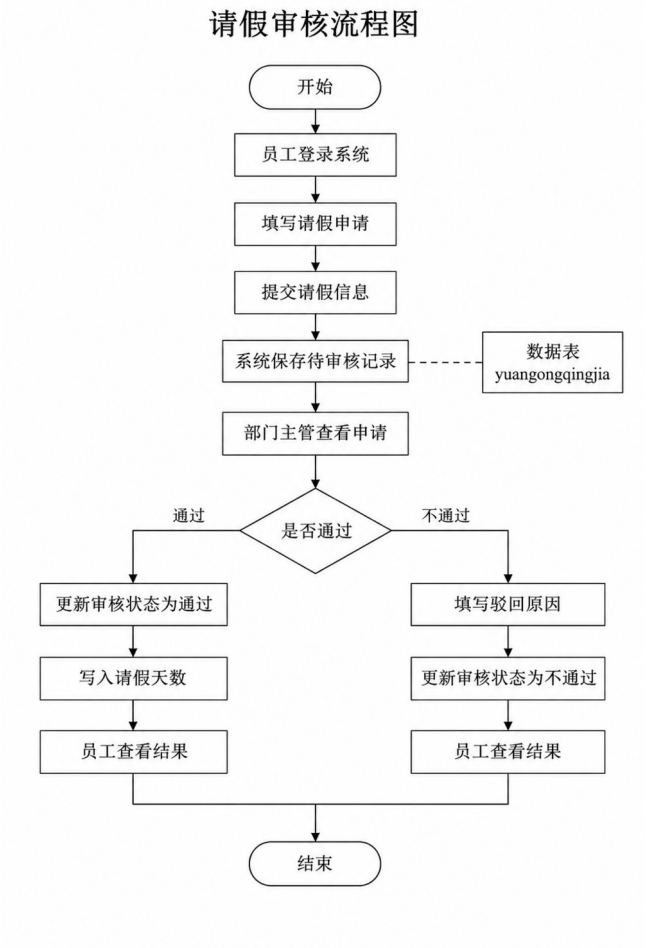

图 5-5 请假审核流程图

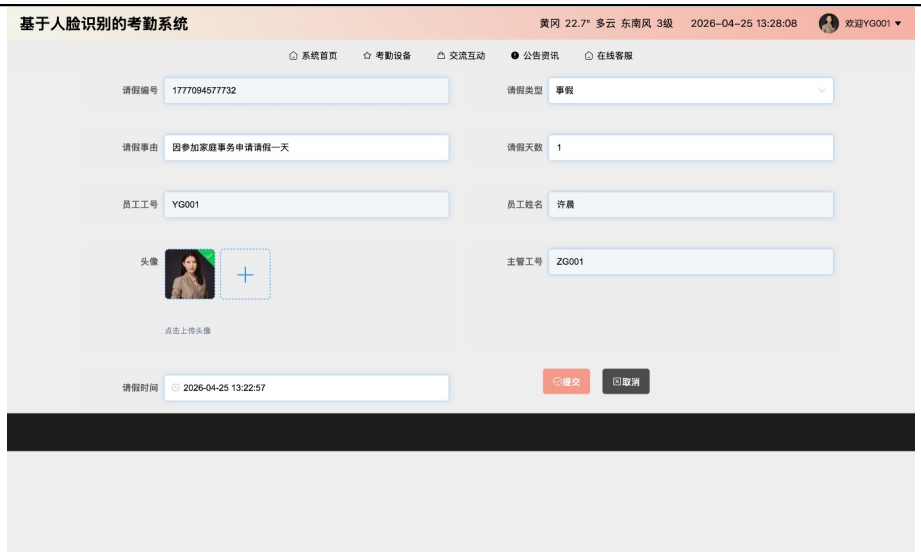

图 5-6 员工请假申请页面

请假模块与考勤记录统计密切相关。若员工请假通过，月度考勤记录中的请假天数应得到体现；若请假未通过，则仍需要按照正常考勤规则进行核查。项目数据库中的请假表包含审核状态和审核回复字段，为后续统计和人工复核提供了依据。

5.6 考勤设备与维保模块实现

考勤设备模块用于维护设备名称、图片、设备位置和所属部门。设备信息既可以在前台供员工查看，也可以在后台由管理员维护。维保记录模块则用于记录设备维护过程，包括维护时间、维护内容、维护人员和备注。对于以设备为入口的考勤系统而言，设备维护信息有助于解释某些时间段的打卡异常。

图 5-7 考勤设备列表页面

项目中设备表和维保表通过设备名称形成业务关联。虽然数据库脚本没有对所有业务关系设置外键约束，但在系统页面和接口逻辑中，设备名称、部门名称等字段承担

了关联查询的作用。论文在总体设计中通过数据库逻辑关系图对这种关系进行了补充说明。

5.7 考勤记录统计模块实现

考勤记录模块用于管理月度出勤结果。与原始打卡记录不同，考勤记录表保存的是统计后的结果，包括迟到次数、早退次数、旷工次数、出差天数、请假天数和实际出勤天数等字段。管理员或部门主管可以根据员工工号、月份和部门名称查询记录，用于后续考勤核对。

从实现角度看，考勤记录模块沿用了项目通用的 Controller、Service、Dao 和 Entity 结构。控制器提供分页查询、列表查询、详情、新增、修改和删除接口，服务层通过 MyBatis-Plus 完成数据访问。由于考勤记录字段较多，页面设计时需要注意字段分组和列表展示宽度，避免统计信息过于拥挤[4,13-17]。

5.8 系统首页与公共信息模块实现

系统首页用于展示系统名称、导航菜单、天气信息和公共入口。公告资讯模块用于发布考勤制度、设备维护通知和系统使用提醒；交流互动和在线客服模块则为用户咨询提供入口。这些模块虽然不直接参与打卡，但能够补充系统的日常管理能力，使平台不只是单一打卡工具。

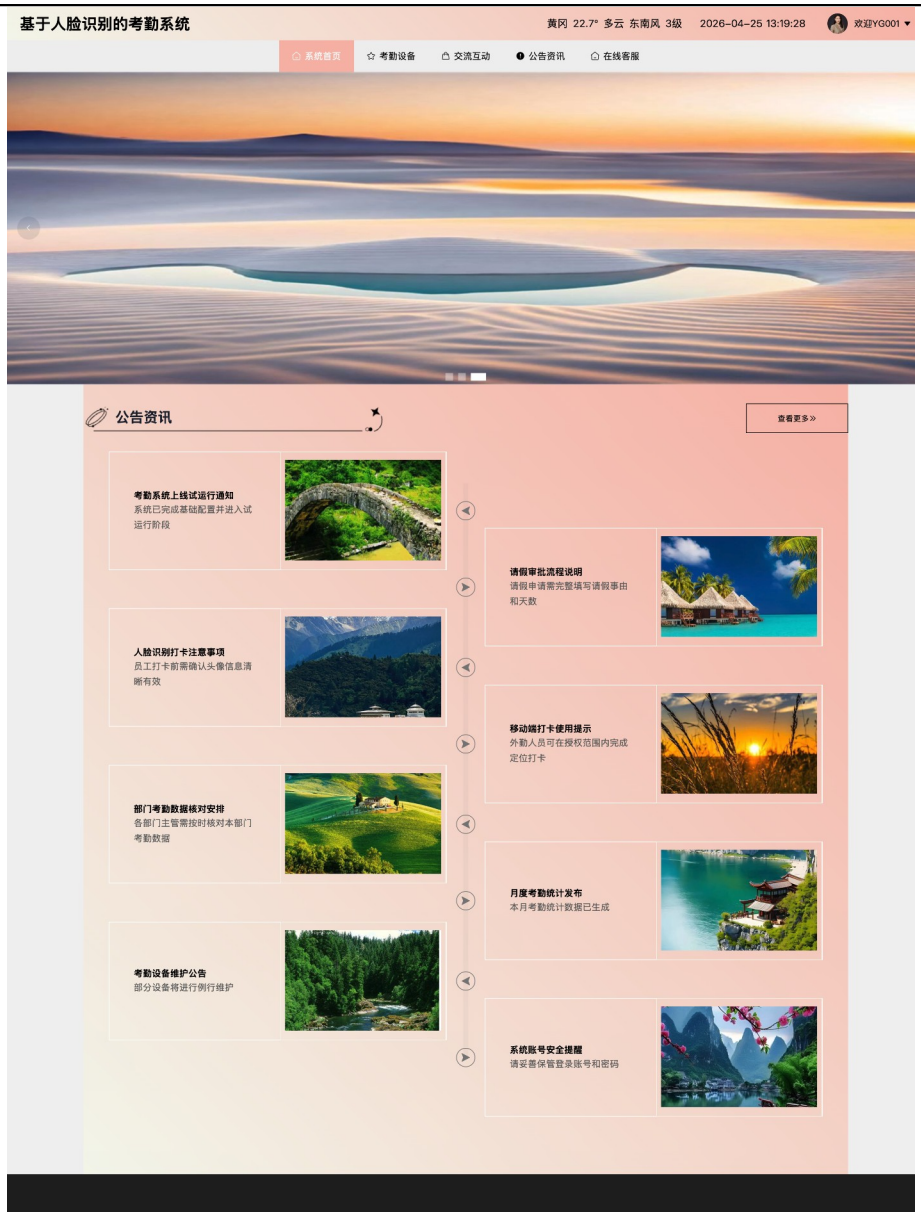

图 5-8 系统首页运行页面

公共信息模块的实现与其他业务模块保持一致，使用数据库表保存资讯内容、分类、封面和发布时间，前端页面负责列表展示和详情查看。统一的实现风格降低了开发复杂度，也让系统具备进一步扩展通知、消息提醒和制度发布功能的基础。

5.9 后台管理模块实现

后台管理模块面向管理员，主要功能包括用户管理、员工管理、部门主管管理、部门管理、考勤设备管理、考勤打卡管理、员工请假管理、考勤记录管理、维保记录管理、公告资讯管理和系统日志查看。后台模块体现了系统对业务数据的集中维护能力，是员工端业务能够正常运行的前提。

图 5-9 为后台考勤打卡管理页面。页面中可以看到不同部门员工的打卡设备、打卡类别、打卡时间、员工工号、员工姓名、部门名称、主管工号和打卡地址等信息，说明系统已经能够基于初始化数据完成后台分页展示和业务核对。

图 5-9 后台考勤打卡管理页面

表 5-1 列出了项目主要实现文件与功能的对应关系。该表能够帮助读者从论文说明追踪到项目源码，体现毕业设计不是脱离代码的文字描述。

表 5-1 主要实现文件说明

文件或目录	对应模块	实现说明
CommonController.java	公共接口	提供文件、人脸比对、数据库导出等公共功能
CommonUtil.java	工具类	封装百度 AI 人脸比对核心逻辑
KaoqindakaController.java	考勤打卡	提供打卡记录查询、新增、修改和删除接口
KaoqindakaEntity.java	考勤打卡实体	映射 kaoqindaka 数据表字段
kaoqindaka/add.vue	前台打卡页面	实现人脸校验、定位采集和打卡提交
YuangongqingjiaController.java	员工请假	实现请假申请和审核相关接口
KaoqinjiluController.java	考勤记录	实现月度考勤记录管理
application.yml	系统配置	配置端口、上下文路径、数据库和 MyBatis 参数

5.10 本章小结

本章结合项目源码和运行页面，对系统主要模块的详细实现进行了说明。系统在人脸识别打卡模块中体现了较强的业务特色，通过前端状态控制、后端公共接口、百度 AI SDK 和定位采集共同完成完整打卡流程。其他模块则围绕人员、部门、设备、请假、记录和公告展开，使系统具备较完整的考勤管理能力。

第 6 章 系统测试

6.1 测试目的与测试环境

系统测试的目的在于验证基于百度 AI 人脸识别的考勤系统是否满足需求分析中提出的功能和非功能要求。测试内容包括登录与权限、员工管理、考勤设备、考勤打卡、人脸识别、定位采集、请假申请、考勤记录、公告资讯和接口异常处理等。由于系统涉及前后端交互和第三方接口调用，测试时既要关注页面操作结果，也要关注接口返回和数据库数据变化[1-3]。

测试环境与项目开发文档保持一致，后端使用 JDK 1.8、Spring Boot、Maven 和 MySQL 5.7，前端使用浏览器访问编译后的 Vue 页面。人脸识别测试需要保证配置表中的百度 AI 访问参数正确，并准备员工头像与打卡图片。定位测试需要浏览器允许位置权限，若精确定位不可用，则观察系统是否进入 IP 定位回退逻辑。

表 6-1 测试环境表

项目	配置	说明
操作系统	Windows/macOS 均可	支持 Java 与浏览器运行环境
后端环境	JDK 1.8、Spring Boot 2.2.2	运行系统后端服务
数据库	MySQL 5.7	导入 springboot5s7063d3.sql
浏览器	Chrome/Edge	访问前台与后台页面
第三方服务	百度 AI、人脸比对接口	提供相似度分数
定位服务	Geolocation + 高德地图	采集经纬度和地址

6.2 功能测试

功能测试采用黑盒测试思路，从用户操作角度检查系统输出是否符合预期。测试用例覆盖登录、打卡、请假、设备、记录和公告等主要业务模块。由于考勤系统的数据具有前后关联性，测试时需要先准备员工、部门、主管和设备信息，再进行打卡和请假测试。

表 6-2 系统功能测试用例

编号	测试功能	测试步骤	预期结果
TC01	用户登录	输入正确账号密码并选择角色登录	登录成功并进入对应页面
TC02	错误登录	输入错误密码连续登录	系统提示登录失败，不进入系统
TC03	员工信息查询	管理员打开员工管理列表并检索员工工号	列表显示符合条件的员工数据
TC04	考勤设备维护	新增设备名称、图片、位置和部门	设备记录保存成功并可在列表查看

TC05	人脸识别打卡	员工上传人脸图片并点击人脸校验	相似度达标时提示匹配成功
TC06	识别失败处理	上传非本人图片进行打卡	系统提示匹配失败并不允许提交
TC07	定位采集	匹配成功后允许浏览器获取位置	经纬度和地址字段被写入表单
TC08	请假申请	员工填写请假类型、事由和天数后提交	请假记录保存为待审核状态
TC09	考勤记录维护	管理员新增或修改月度考勤记录	记录保存成功，列表展示更新结果
TC10	公告资讯	管理员发布公告并在前台查看	前台公告列表显示新公告

6.3 人脸识别与定位测试

人脸识别测试重点检查系统能否正确调用后端接口并根据返回分数控制打卡状态。测试时准备员工头像作为基础图，再上传同一人员或不同人员的人脸图片。若为同一人员，系统应返回较高相似度并提示匹配成功；若为不同人员或图片无效，系统应返回低分或错误提示。前端必须保证识别失败时不能绕过校验直接提交。

表 6-3 人脸识别测试用例

编号	测试输入	操作过程	预期结果
FR01	员工本人头像与本人打卡照	调用 matchFace 接口并观察 score	score 大于 60，允许继续定位
FR02	员工头像与非本人图片	上传非本人图片后点击校验	score 不达标，提示匹配失败
FR03	打卡图片为空	不上传图片直接校验	提示请上传人脸图片
FR04	配置项错误	服务端无法获取有效密钥	返回接口错误信息，不保存打卡
FR05	图片文件不存在	传入不存在的文件路径	后端返回文件不存在或错误结果

定位测试重点检查浏览器定位和 IP 定位回退逻辑。当用户允许浏览器获取位置时，系统应写入经纬度和格式化地址；当浏览器拒绝授权或定位超时时，系统应提示尝试 IP 定位，并尽量写入城市级地址。定位测试不要求每次获得完全一致的地址，但需要保证表单字段能够被正确更新。

6.4 接口测试

接口测试主要检查后端控制器是否能够按照预期返回数据。项目各业务接口大多采用统一返回格式，成功时返回 code 为 0 并携带 data，失败时返回错误消息。分页接口需要检查 page、limit、sort 和 order 等参数是否生效，新增和修改接口需要检查数

据库记录是否对应变化。

表 6-4 主要接口测试用例

接口	测试内容	预期结果
/kaoqindaka/page	携带分页参数查询打卡记录	返回分页数据和总数
/kaoqindaka/add	提交完整打卡记录 JSON	返回新增记录 id
/yuangong/session	登录后获取当前员工信息	返回员工工号、姓名、部门等信息
/yuangongqingjia/add	提交请假申请	请假记录保存成功
/kaoqinjilu/page	按月份或员工查询考勤记录	返回符合条件的数据
/file/upload	上传图片文件	返回 upload 路径
/matchFace	传入两张图片路径	返回 score 相似度字段

6.5 权限与异常测试

权限测试用于验证不同角色是否只能访问其权限范围内的功能。员工不应直接管理全体员工和系统日志，部门主管应主要查看本部门相关信息，管理员能够访问后台全部管理模块。项目中的部分接口会根据 session 中的 username 和 tableName 设置查询条件，这一点在测试时需要重点关注。

异常测试包括空字段提交、错误图片路径、重复部门名称、非法分页参数和网络失败等情况。系统不应在异常情况下出现页面崩溃或后端未捕获错误，而应尽量给出明确提示。对于百度 AI 接口异常，系统当前采用返回 score 为 0 或错误信息的方式处理，能够避免异常识别结果继续进入考勤提交环节。

6.6 测试结果分析

从测试结果看，系统主要功能能够按照预期运行。登录模块能够区分用户身份；考勤设备、员工、部门和考勤记录等基础模块能够完成常规管理；人脸识别打卡流程能够在识别通过后进入定位和提交环节；请假申请模块能够保存申请数据；公告和交流模块能够补充系统信息发布能力。整体测试结果表明系统功能基本满足需求。

系统仍存在可进一步完善之处。例如当前人脸比对阈值固定为 60，后续可以根据实际采集环境做动态调整；定位信息依赖浏览器权限，若用户拒绝授权，只能使用 IP 定位；数据库表之间部分关系依赖业务字段而非外键约束，后续可结合业务需要增加约束或索引。总体而言，这些问题不影响系统主要流程运行，但可作为后续优化方向。

6.7 本章小结

本章对系统进行了功能测试、接口测试、人脸识别测试、定位测试、权限测试和异常测试。测试结果表明，系统能够实现预期功能，核心的人脸识别考勤流程具有可用性。通过测试也发现了阈值、定位和数据约束方面的改进空间，为下一章总结与展望

提供依据。

第 7 章 总结与展望

7.1 工作总结

本文围绕基于百度 AI 人脸识别的考勤系统完成了需求分析、总体设计、详细实现和系统测试。系统采用 Spring Boot 作为后端框架，Vue 构建前台和后台页面，MyBatis-Plus 完成数据访问，MySQL 保存业务数据，并通过百度 AI 人脸比对接口提升打卡身份核验能力。论文在写作过程中结合项目源码、数据库脚本和运行页面截图，对系统进行了较完整的工程化说明[4,9-10]。

在需求分析方面，本文明确了管理员、员工和部门主管三类角色的业务边界，梳理了员工管理、部门管理、考勤设备、考勤打卡、考勤记录、员工请假、维保记录、公告资讯和系统日志等功能。在总体设计方面，本文给出了系统架构图、部署图、包图、人脸识别打卡流程图、E-R 图和数据库逻辑关系图，并对主要数据表和接口进行了说明。

在详细实现方面，本文重点分析了登录权限、人脸识别、定位打卡、请假审核、考勤设备、考勤记录和后台管理等模块。通过源码可以看出，系统将百度 AI SDK 封装在后端工具方法中，前端只根据接口返回 score 控制业务流程，避免将敏感配置暴露在浏览器端。通过测试可以看出，系统主要功能能够正常运行，基本满足考勤管理的应用需求。

7.2 不足之处

由于系统属于毕业设计项目，仍存在一些不足。第一，人脸识别主要采用图片比对方式，尚未加入严格的活体检测机制，如果用户提交照片翻拍图片，仍可能存在识别风险。第二，定位模块依赖浏览器权限和网络环境，在复杂室内环境下可能出现偏差。第三，考勤统计规则目前以字段维护为主，后续可以加入更加自动化的统计任务，例如根据上下班时间自动判断迟到和早退。

第四，数据库中部分业务关系没有完全使用外键约束，更多依靠字段名称保持一致。虽然这种方式方便项目快速开发，但在数据规模扩大后可能出现维护成本增加的问题。第五，系统界面仍以常规管理页面为主，后续可以增加图表统计、异常提醒和移动端适配，提高管理人员查看考勤情况的效率。

7.3 展望

后续系统可以从三个方向继续完善。首先，在识别安全方面，可以接入活体检测、多人脸检测和图片质量检测，避免低质量图片或非本人照片通过识别。其次，在考勤统计方面，可以增加自动统计任务，根据打卡时间、请假记录和节假日规则生成月度考勤结果，并支持导出 Excel。再次，在应用形态方面，可以开发移动端或小程序端，使员工在手机上完成打卡、请假和消息查看。

此外，系统还可以增加异常考勤预警功能。当员工连续迟到、同一地点短时间多次异常打卡或设备长期未维护时，系统自动向主管或管理员发送提醒。随着数据量增加，还可以从历史考勤数据中分析部门出勤趋势，为管理决策提供辅助。总体来看，本文

实现的系统已经具备较完整的基础功能，后续仍具有较好的扩展空间。

参考文献

[1] 仝珊.基于人脸识别的考勤签到系统实现[J].电脑编程技巧与维护,2021:4.

[2] 黄勇.基于人脸识别的学校考勤系统设计[J].电脑编程技巧与维护,2023:4.

[3] 李宗海;张欢欢.基于 iOS 人脸识别考勤系统的设计与实现[J].软件,2022:3.

[4] 陶琳.基于 Spring Boot 和 Vue 框架的高校实验室耗材管理系统的分析与设计[J].电脑知识与技术,2021:3.

[5] 杨晟;罗奇.基于 Spring Boot 的在线影院售票系统的设计[J].网络安全技术与应用,2022:2.

[6] 丁润东;王亚;肖念昕;张志扬;周子康;马唯一.基于 Spring Boot 的食品企业 ERP 管理系统设计与实现[J].电脑知识与技术,2022:4.

[7] 杨明芬,李文雯,吴旭.基于 Spring Boot 的西藏遥感信息服务系统设计与实现[J].西藏科技,2023:5.

[8] 姚居文.基于 Spring Boot 微服务框架的职业院校技能大赛报名系统设计[J].电脑知识与技术,2022:5.

[9] 史远航;饶欣宇;解佳坤;李芒;佟宽章.基于百度 AI 的人脸识别新生报到系统设计与实现[J].网络安全技术与应用,2021:3.

[10] 王以伍,舒晖.基于 SpringBoot+Vue 前后端分离的高校实验室预约管理系统的设计与实现[J].现代计算机,2023:4.

[11] 单树倩;任佳勋.基于 SpringBoot 和 Vue 框架的数据库原理网站设计与实现[J].电脑知识与技术,2021:3.

[12] 蔡文红;张剑;王玉业.基于 Vue.JS 的医院招标采购数字化内控管理系统[J].电脑采购,2021:3(166-168).

[13] 佟羽齐,刘德华,张栎柯.基于 SpringBoot 框架的钻前管理系统的设计与实现[J].信息与电脑(理论版),2023:3.

[14] 刘子凡;郭昱君.基于 SpringBoot+Mybatis 的个人博客系统设计与实现[J].现代信息科技,2021:5.

[15] 李方方;夏新澳.基于 JPlag 与 Springboot 的代码查重云系统[J].信息技术与信息化,2021:3.

[16] 赵胤章.基于 SpringBoot 的立案登记系统研究[J].无线互联科技,2022:3.

[17] 王亚东,李晓霞,陈强强.基于 SpringBoot 的需求发布平台设计[J].信息与电脑(理论版),2023:3.

[18] 巢晟盛.基于 SpringBoot 微服务架构下前后端分离的 MVVM 模型浅析[J].电脑知识与技术,2021:3.

[19] 夏凡华.Go 前后端分离 Web 开发的实现[J].电脑编程技巧与维护,2021:4.

附　录

附录列出系统核心代码片段，主要包括人脸比对接口、百度 AI 比对工具方法和前台打卡控制逻辑。为避免泄露实际配置，代码中的密钥值不在论文中展示。

附录 A-1 公共人脸比对接口

```
@RequestMapping("/matchFace")
@IgnoreAuth
public R matchFace(String face1, String face2, HttpServletRequest request) {
    return CommonUtil.matchFace(face1, face2);
}
```

附录 A-2 百度 AI 人脸比对核心方法节选

```
String apiKey = configService.selectOne(
        new EntityWrapper<ConfigEntity>().eq("name", "APIKey")).getValue();
String secretKey = configService.selectOne(
        new EntityWrapper<ConfigEntity>().eq("name", "SecretKey")).getValue();
AipFace client = new AipFace(null, apiKey, secretKey);
String img1 = Base64Util.encode(FileUtil.FileToByte(file1));
String img2 = Base64Util.encode(FileUtil.FileToByte(file2));
ArrayList<MatchRequest> requests = new ArrayList<MatchRequest>();
requests.add(new MatchRequest(img1, "BASE64"));
requests.add(new MatchRequest(img2, "BASE64"));
JSONObject res = client.match(requests);
return R.ok().put("score", JSONObject.parse(
        res.getJSONObject("result").get("score").toString()));
```

附录 A-3 前台打卡提交逻辑节选

```
if (!this.faceMatchFlag) {
    const res = await this.$http.get("matchFace", { params: params });
    if (res.data && res.data.code == 0 && res.data.score > 60) {
        this.faceMatchFlag = true;
        this.$message({ message: "□□□□", type: "success" });
    } else {
        this.faceMatchFlag = false;
        this.$message.error("□□□□");
    }
    await this.forceGetCurrentPosition();
    return;
}
await this.$http.post(`kaoqindaka/${this.ruleForm.id ? "update" : "add"}`,
        this.ruleForm);
```

致　谢

本论文从选题、需求梳理、项目分析到论文撰写，均得到了老师和同学的帮助。在系统设计过程中，指导教师对论文结构、技术路线和格式规范提出了宝贵意见，使本文能够围绕真实项目展开，而不是停留在简单功能罗列层面。在此向指导教师表示诚挚感谢。

同时感谢同学在系统运行、资料整理和问题讨论中给予的帮助。通过本次毕业设计，我进一步理解了 Spring Boot、Vue、数据库设计和第三方人工智能接口接入的完整流程，也认识到软件工程项目不仅要实现功能，还要重视需求分析、结构设计、测试验证和文档表达。今后将继续加强工程实践能力，提高系统设计与问题解决水平。